

机器学习任务攻略

李宏毅《机器学习》2025 · 第二节 (General Guidance)



基于李宏毅老师课程整理

2026 年 6 月 4 日

视频信息

频道：李宏毅深度学习课堂 (Bilibili 搬运)

时长：约 51 分钟

链接：<https://www.bilibili.com/video/BV1TAtwzTE1S?p=3>

目录

1 引言：作业通关的总攻略	3
1.1 所有作业的共同格式	3
1.2 训练的三个步骤（回顾）	3
1.3 攻略全景图：一棵诊断决策树	4
2 训练损失大：Model Bias 还是 Optimization 失败	5
2.1 Model Bias：模型太简单，针不在海里	6
2.2 Optimization 失败：针在海里，却捞不出来	7
2.3 如何区分二者：比较不同的模型	7
2.3.1 ResNet 的经典实验	7
2.3.2 实用建议与观看人数例子	8
3 训练损失小、测试损失大：Overfitting	8
3.1 为什么会发生 Overfitting	9
3.2 解法一：增加训练资料 / Data Augmentation	10
3.3 解法二：限制模型（但别限制过度）	10
3.4 Bias-Complexity Trade-off	11
4 模型选择 (Model Selection)	12
4.1 为什么不能用 public 测试集来挑模型	12
4.2 正确做法：切出 Validation Set	13
4.3 N-fold 交叉验证	13
5 回到观看人数预测：Mismatch	14
5.1 2 月 26 日的预测结果	14
5.2 Mismatch：训练与测试分布不同	16
6 总结与延伸	16
6.1 下一讲预告	17

1 引言：作业通关的总攻略

从这一节开始，李宏毅老师不再讲单一模型，而是给出一套**适用于前期所有作业的通用攻略** (General Guidance)。老师把它比喻成游戏里的开局神装——“就跟魔关羽一样，开局就送，可以帮你打赢前期所有的副本”。

1.1 所有作业的共同格式

之后好几个作业看起来都大同小异：你会拿到一堆**训练资料**，每笔含输入 x 与对应的标签 $\hat{y}$ ；而**测试资料**只有 x 、没有 $\hat{y}$ ，需要你的模型去预测。

各作业的 $X \rightarrow Y$ 一览

- **作业 1 (回归)**: X = 过去的疫情/观看数据, Y = 未来某天的数值
- **作业 2 (语音辨识)**: X = 一小段声音讯号, Y = 对应的音素 (phoneme, 可暂时理解为 KK 音标)
- **作业 3 (影像辨识)**: X = 一张图片, Y = 图片里是什么东西
- **作业 4 (语者辨识)**: X = 一段声音讯号, Y = 说话的是哪一个人 (银行客服的身份验证就用这种系统)
- **作业 5 (机器翻译)**: X = 一种语言的句子, Y = 另一种语言的句子

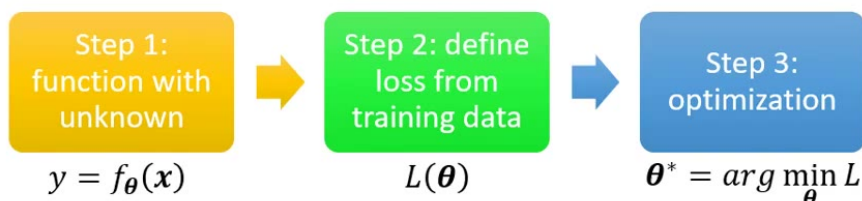
1.2 训练的三个步骤 (回顾)

训练资料拿来训练 model, 过程就是上一节讲的三个步骤:

Framework of ML

Training data: $\{(x^1, \hat{y}^1), (x^2, \hat{y}^2), \dots, (x^N, \hat{y}^N)\}$

Training:



Testing data: $\{x^{N+1}, x^{N+2}, \dots, x^{N+M}\}$

Use $y_{\circ} = f_{\theta^*}(x)$ to label the testing data

$\{y^{N+1}, y^{N+2}, \dots, y^{N+M}\}$

图 1: 机器学习框架: Step 1 写出带未知参数的函数 $y = f_{\theta}(x)$; Step 2 从训练资料定义损失 $L(\theta)$; Step 3 解最佳化问题求 $\theta^* = \arg \min_{\theta} L$ 。最后用 f_{θ^*} 去标注测试资料。

1. 写出一个有未知数的函数 $y = f_{\theta}(x)$, 其中 θ 代表 model 里所有未知参数, 输入 x 称为 feature
2. 定义损失函数 $L(\theta)$, 衡量一组参数有多好/多坏
3. 解最佳化问题, 找出让损失最小的 θ^* , 再把它代回函数用于测试资料

直接执行助教的 sample code, 往往只能通过 simple baseline。想做得更好, 就要按下面这张攻略图逐步诊断。

1.3 攻略全景图: 一棵诊断决策树

整套攻略的核心, 是一棵从损失出发的决策树。后续每一节都是在走这棵树的某一条分支, 建议先把它印在脑海里:

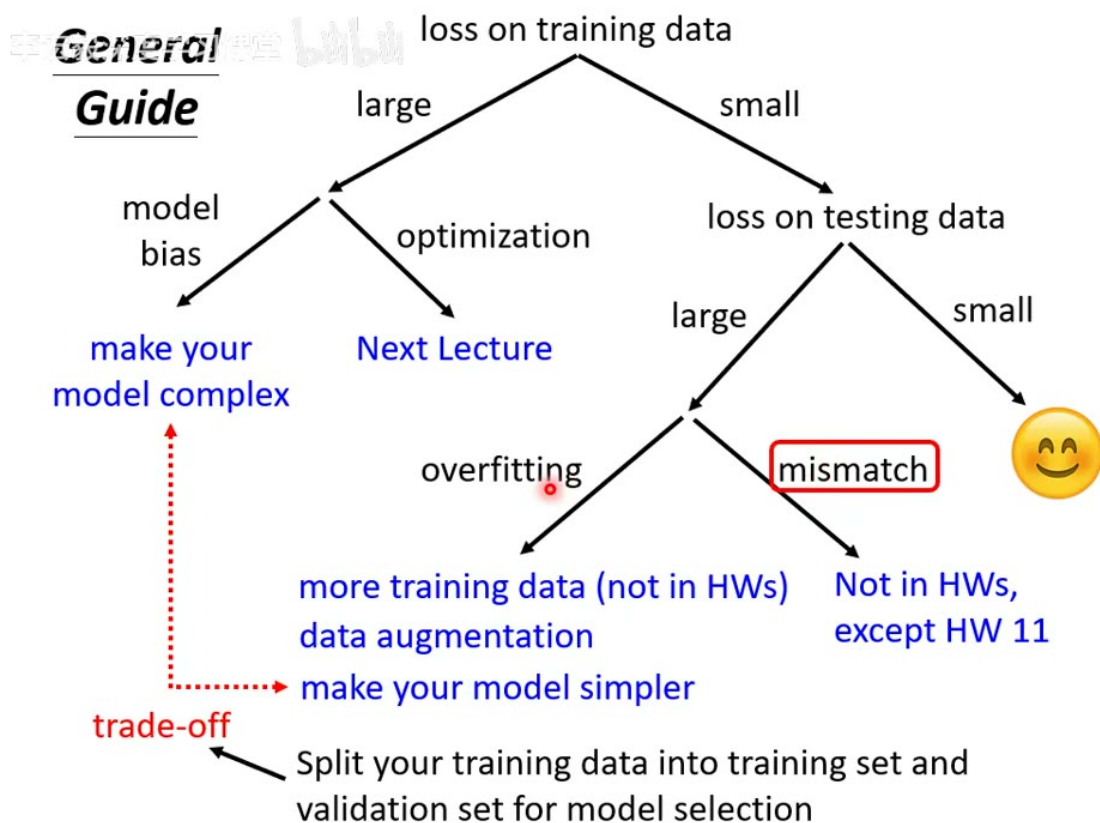


图 2: 机器学习任务攻略决策树 (General Guide): 先看**训练资料**上的损失, 再看**测试资料**上的损失, 沿不同分支对症下药。

攻略的核心顺序：先看训练损失，再看测试损失

很多同学一看到 Kaggle 上（测试资料）结果不好，就直接下结论说“overfitting”。**这是最常见的错误**。正确的第一步永远是：

先检查 **训练资料** 的 loss，确认 model 在训练资料上“学起来”了，再去查看测试资料。

决策树主干：

- 训练 loss 大 → 是 Model Bias 还是 Optimization 失败？（第 2 节）
- 训练 loss 小、测试 loss 大 → 是 Overfitting 还是 Mismatch？（第 3、5 节）
- 训练 loss 小、测试 loss 也小 → 恭喜通关（:-）

2 训练损失大：Model Bias 还是 Optimization 失败

如果训练资料上的 loss 就很大，说明 model 连训练资料都没学好。原因有两个：**Model Bias** 与 **Optimization 失败**。

2.1 Model Bias: 模型太简单，针不在海里

Model Bias

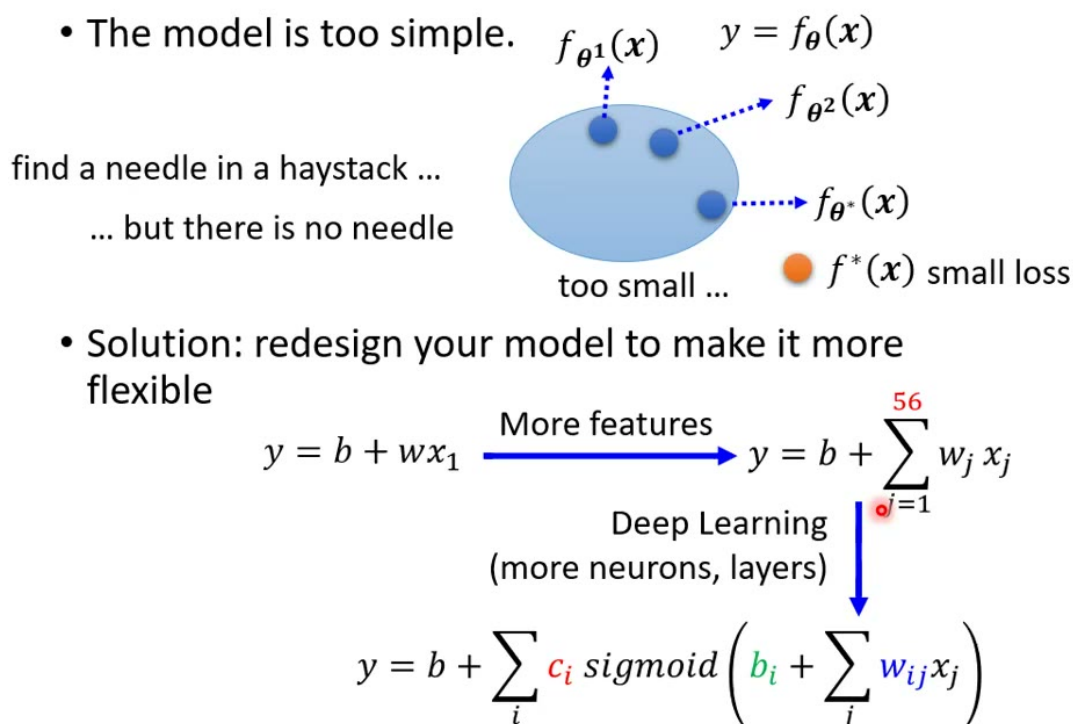


图 3: Model Bias: model 能表示的所有函数构成一个集合 (function set), 但这个集合“太小”, 里面根本没有能让 loss 变低的函数 (橙点不在蓝色集合内)。解法是重新设计更有弹性的 model。

把不同参数 θ 代入函数, 会得到不同的函数; 把它们全部集合起来, 就是这个 model 能表示的 function set。Model Bias 指的是: 这个集合太小, 里面没有任何一个函数能让 loss 足够低。

大海捞针的比喻

寻找低 loss 的函数, 就像在大海里捞针。Model Bias 的情况是: 针根本不在海里。所以无论 optimization 多努力, 找到的 θ^* 也只是这些差函数里“最好的那一个”——“它只是鲁蛇里面的霸主, 还是一个鲁蛇”, loss 依然不够低。

解法: 重新设计、给 model 更大的弹性。 承接上一节的做法:

$$y = b + wx_1 \xrightarrow{\text{增加 feature}} y = b + \sum_{j=1}^{56} w_j x_j \xrightarrow{\text{Deep Learning}} y = b + \sum_i c_i \sigma \left(b_i + \sum_j w_{ij} x_j \right)$$

即: 增加输入的 feature 数量、或用更多神经元/更多层的深度网络来扩大 function set。

再次提醒: Model Bias 的 “bias” 不是参数 b

Model Bias 是“模型结构的局限”, 与函数里那个截距参数 b (bias) 完全是两回事。这一点上一节已强调过, 这里再次确认, 避免混淆。

2.2 Optimization 失败：针在海里，却捞不出来

并不是训练 loss 大就一定是 Model Bias。另一种可能是 **optimization 没做好**。本课程的 optimization 方法只用 gradient descent，而它有很多问题——比如会卡在 local minima，找不到真正能让 loss 很低的参数。

两种情况的比喻对比

- **Model Bias**：针不在海里（function set 里没有好函数）→ 白忙一场
- **Optimization 失败**：针在海里，但 gradient descent 没本事把它捞出来（好函数存在，却找不到）

2.3 如何区分二者：比较不同的模型

看到训练 loss 大，怎么知道是“海里没针”还是“捞不出针”？**建议的方法：比较不同复杂度的模型。**

2.3.1 ResNet 的经典实验



Ref: <http://arxiv.org/abs/1512.03385>

Model Bias v.s. Optimization Issue

- Gaining the insights from comparison

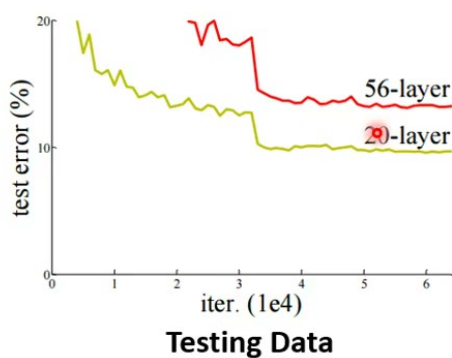


图 4: ResNet 论文 (2015, arXiv:1512.03385) 的实验：在测试资料上，56 层网络（红）的 error 反而比 20 层（黄）更高。

当年很多人看到这张图就说“56 层太深、overfitting 了，所以深度学习不 work”。但这并不是 **overfitting**。检查训练资料会发现：56 层在训练资料上的 loss 也比 20 层高。

为什么这是 Optimization 失败，而非 Model Bias 或 Overfitting

- 不是 Overfitting：因为它连训练资料的 loss 都更高（overfitting 是训练好、测试差）
- 不是 Model Bias：56 层的弹性一定 $\geq$ 20 层。56 层只要让前 20 层和那个 20 层网络一样、后 36 层全做 identity（原样输出），就能复制 20 层的结果
- 所以是 Optimization 失败：56 层有能力做得更好，gradient descent 却没找到那组参数

2.3.2 实用建议与观看人数例子

诊断 Optimization 的实用建议

遇到一个没做过的新问题，可以先跑比较浅、比较小的网络，甚至非深度学习的方法（如 linear model、SVM）。这些模型“会竭尽全力在能力范围内找出最好的参数”，比较不会有 optimization 失败的问题，可以作为 baseline。

之后再跑深的 model：如果深的 model 弹性更大、loss 却压不过浅的 model，那就说明 optimization 出了问题。

上一节的观看人数预测实验正是一例（训练资料为 2017–2020 年）：

表 1: 不同层数网络在训练资料上的 loss（单位：千人次）

层数	1 层	2 层	3 层	4 层	5 层
训练 loss	0.28k	0.18k	0.14k	0.10k	0.34k

1→4 层 loss 一路递减，到 5 层却暴增到 0.34k。这显然不是 Model Bias（4 层都能做到 0.10k，弹性更大的 5 层没理由更差），而是 optimization 没做好。至于 optimization 失败具体怎么解决，留待下一节课（local minima 与 saddle point）。

3 训练损失小、测试损失大：Overfitting

假设你已经让训练 loss 变小了，接着看测试 loss。若测试 loss 也小、且小于 strong baseline，那就通关了。但若训练 loss 小、测试 loss 大，这才可能是真正的 Overfitting。

Overfitting 的严格定义

训练 loss 小 + 测试 loss 大 = Overfitting。两个条件缺一不可。老师说：“十个同学有八个”被问到“训练 loss 多少”时都答不上来——请务必先记录训练 loss，确认 optimization 没问题、model 够大之后，再来谈 testing 的问题。

3.1 为什么会发生 Overfitting

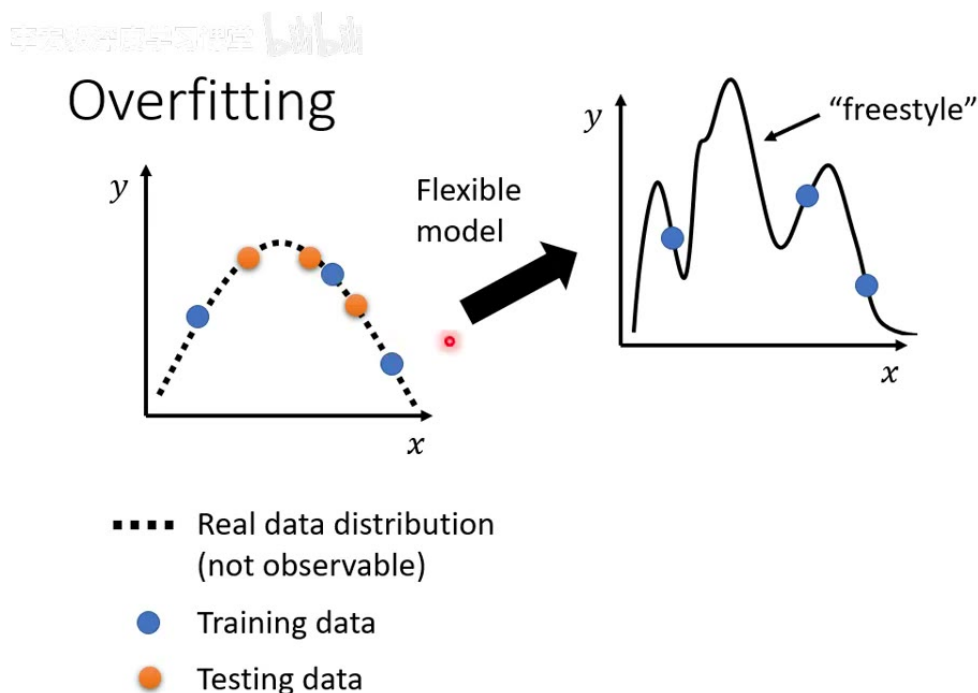


图 5: Overfitting 直觉: 真实关系是虚线 (不可观测), 训练资料只是其上 sample 出的几个蓝点。弹性极大的 model 会精确穿过训练点, 但在没有数据约束处 “freestyle” 乱长, 导致测试点 (橙) 上误差很大。

一个极端的“一无是处”函数

设想一个很废的函数: 输入 x 时, 先查它有没有出现在训练资料里——有就输出对应的 $\hat{y}$, 没有就输出一个随机值。

这个函数啥也没学到, 但它在训练资料上 $\text{loss} = 0$ (全背下来了), 在测试资料上 loss 极大 (全靠乱猜)。这就是 overfitting 的极端形态。

一般情况下, 当 model 弹性 (自由度) 很大时, 只用少数几个训练点作约束, 其余地方它可以产生各式各样奇怪的曲线, 于是训练好、测试差。

3.2 解法一：增加训练资料 / Data Augmentation

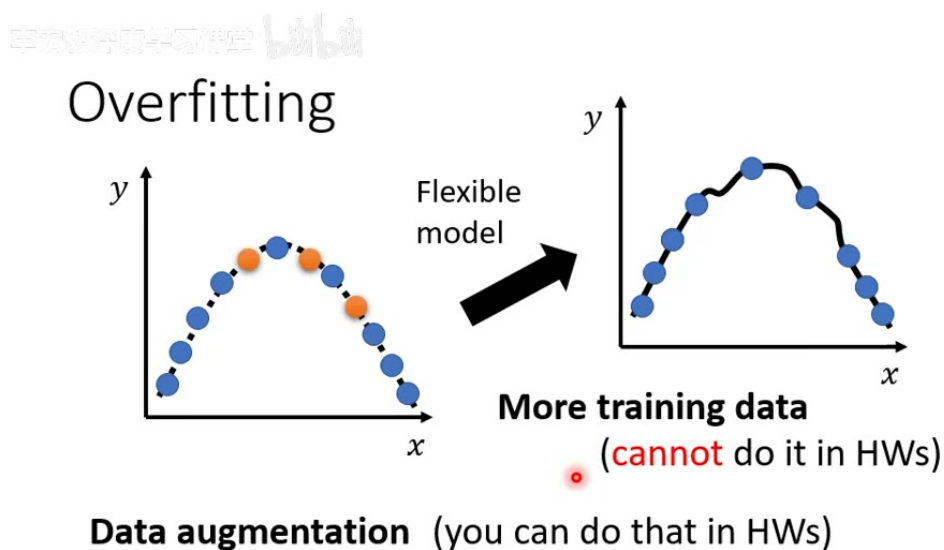


图 6: 增加训练资料: 蓝点变多后, 即使 model 弹性很大, 也会被密集的数据“限制住”, 形状被迫贴近真实的二次曲线。

增加训练资料往往是最有效的解法。但作业里不允许自己额外收集资料（以免大家把精力花在搜集资料而非机器学习核心技术上）。作业里能用的是 **Data Augmentation**:

Data Augmentation (资料增强)

用你对问题的理解, 从已有资料创造出新资料——这不算使用额外资料。例如影像辨识中把图片左右翻转、或截出一块放大, 资料量就翻倍。

但要 **augment** 得有道理: 影像辨识中很少把图片上下颠倒, 因为那不是真实世界会出现的画面, 机器看了反而会学到奇怪的东西。

3.3 解法二：限制模型（但别限制过度）

另一个方向是不要给 model 那么大的弹性、给它一些限制, 最好让 model 正好契合产生资料背后的规律（例如已知是二次曲线, 就限制 model 只能是二次曲线）。

给 model 制造限制的常见手段

- 给比较少的参数 / 比较少的神经元
- 共用参数（让一些参数取相同数值）——CNN 就是典型
- 用比较少的 feature
- Early Stopping、Regularization、Dropout（后续课程详讲；作业一的 sample code 里已含 early stopping）

为什么 CNN 在影像上反而更好？

Fully-connected network 是弹性较大的架构，它能表示的 function set 较大；CNN 是受限的架构，function set 较小、且包含于 fully-connected 之内。但正因为 CNN 针对影像特性施加了合理的限制，它在影像问题上反而做得更好。“弹性大”不等于“更好”。

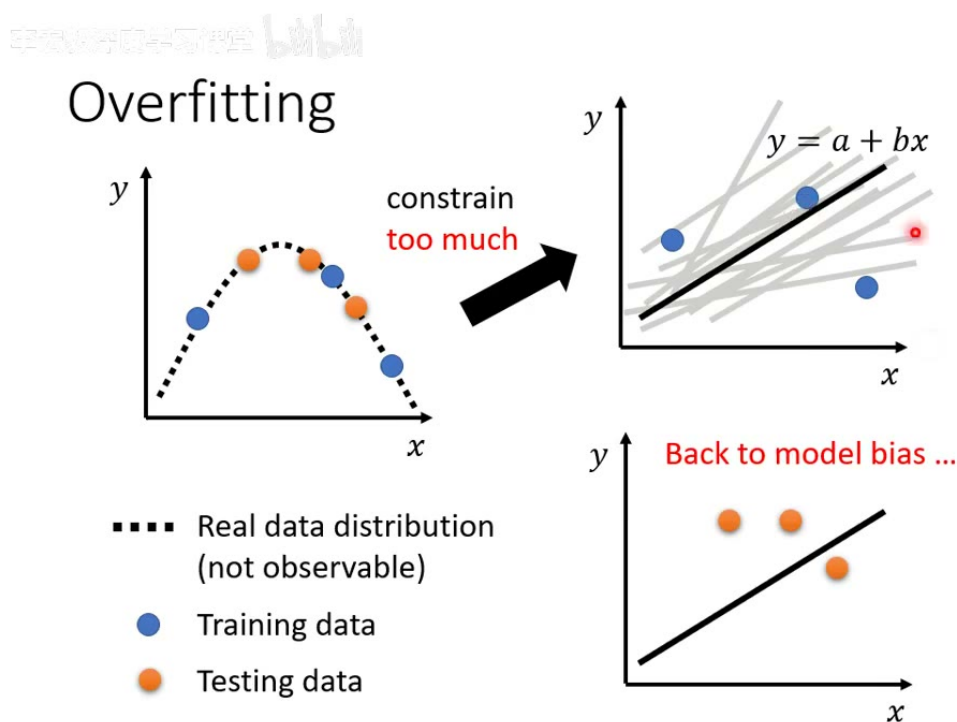


图 7：限制过度的反例：若强行规定 model 只能是直线 $y = a + bx$ ，则没有任何直线能同时通过这三个点，测试结果同样很差——但这不是 overfitting，而是回到了 Model Bias。

限制不能给太多——否则回到 Model Bias

给的限制太多、model 太死板，就会重新陷入 Model Bias：结果不好不是因为 overfitting，而是因为 function set 又变得太小、装不下好函数。

3.4 Bias-Complexity Trade-off

于是出现一个矛盾：model 太简单有 Model Bias，太复杂有 Overfitting。

Bias-Complexity Trade-off

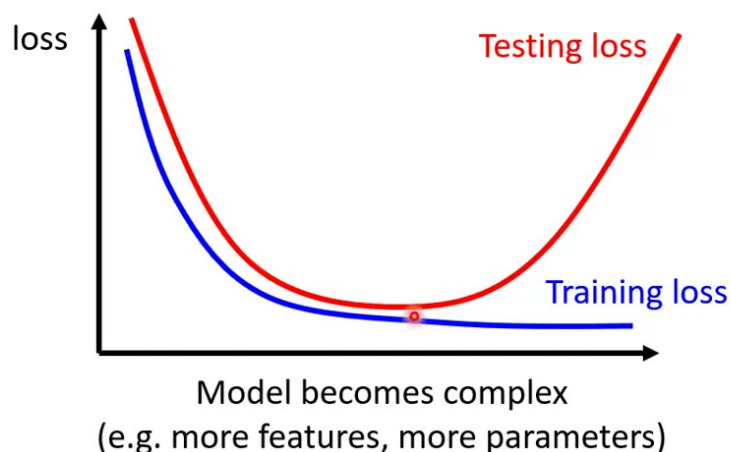


图 8: Bias-Complexity Trade-off: 随 model 复杂度 (更多 feature/参数) 增加, 训练 loss 单调下降; 而测试 loss 先降后升, 呈 U 形。最优点在中间“不偏不倚”处。

复杂度与损失的关系

- **训练 loss**: 随复杂度增加**单调下降**
- **测试 loss**: 先随复杂度下降, 超过某临界点后因 overfitting **突然回升** (U 形)

我们想选一个**中庸的 model**——不太复杂也不太简单, 恰好在测试 loss 最低处。怎么选? 见下一节。

(“复杂度/弹性”在本节只是直观描述, 其严格定义将在后续“吴沛元老师”的课程中给出。)

4 模型选择 (Model Selection)

4.1 为什么不能用 public 测试集来挑模型

最直觉的做法: 把几个模型的结果都传上 Kaggle, 看哪个分数最低就选哪个。**老师强烈不建议这么做。**

一兆个废模型的极端例子

假设有一兆个“一无是处”的模型, 每个在测试资料上都是**随机输出**。把它们的结果全传上 Kaggle, 总会有某一个 (比如编号 56789) **碰巧**在 public 测试集上得到好分数。你以为它是好 model, 其实它只是随机蒙中。

后果: **public 排前十, private 一公布就崩**——掉到 300 名甚至 1000 名之外。“这件事每年都会发生, 并不是传说。”

Benchmark 上“超越人类”的真相

这也解释了为什么在公开的 benchmark corpus (如语音辨识的 LibriSpeech、Switchboard) 上, 机器常“超越人类”——因为测试集是**公开的**, 只要刷得够多, 再废的模型也能在上面蒙到好成绩。2016 年微软、IBM 宣称语音辨识超越人类, 就是 benchmark 上的结果; 现实生活中 (private 场景) 显然没有。这类宣传“比较像是骗骗麻瓜的商业实力”。

为此, Kaggle 把测试集分成 public 与 private, 且**限制每日上传次数**, 就是为了防止大家靠不断随机上传来碰运气。

4.2 正确做法: 切出 Validation Set

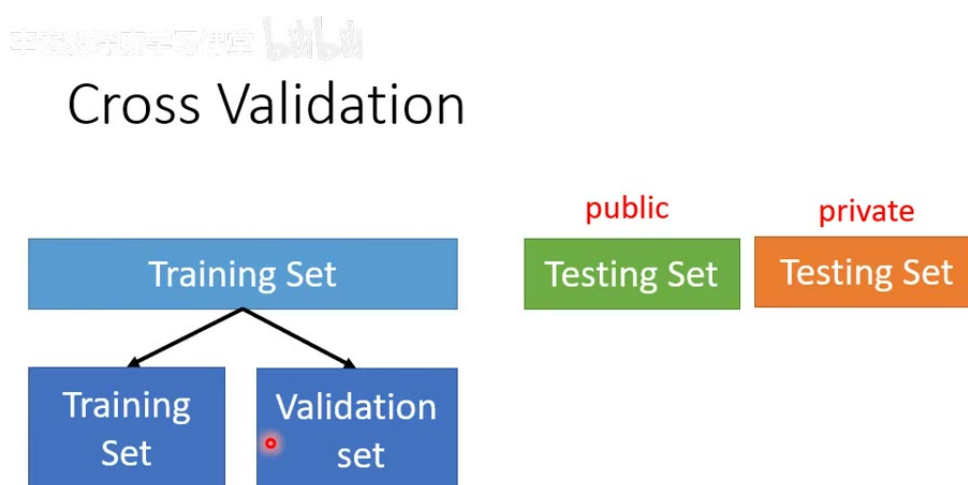


图 9: 把训练资料再切成 Training Set 与 Validation Set (如 90% / 10%)。在 Training Set 上训练、在 Validation Set 上衡量并**挑选**模型, 再上传。

用 Validation Set 挑模型

用 validation set 上的分数来选 model, 则 public 测试集的分数就能较好地反映 private 测试集的分数, 不会出现“public 极好、private 极差”的悲剧。

理想做法: 用 validation loss 最小的那个直接定案, **尽量不要再根据 public 分数去反复调整模型**——看 public 结果调得越多, 就越可能 fit 在 public 上, private 又会崩。

4.3 N-fold 交叉验证

如果担心“万一 validation set 恰好分得很差”, 可以用 **N-fold Cross Validation**。

N-fold Cross Validation

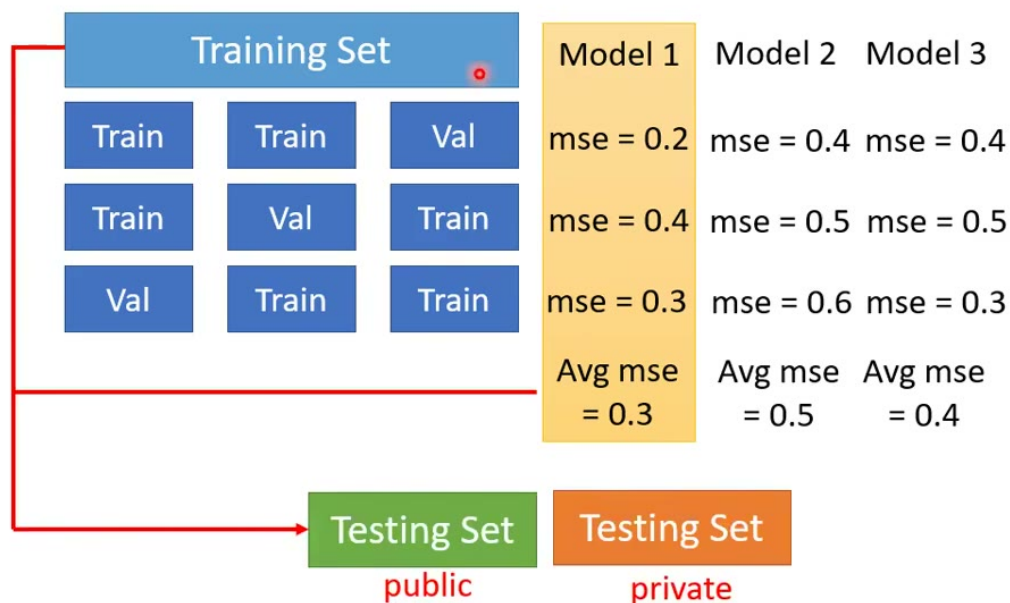


图 10: N-fold 交叉验证 (此处 $N = 3$): 训练资料切成 3 份, 轮流让其中一份当 Validation、其余当 Training, 共 3 种切法。每个模型在 3 种切法下各跑一次, 取平均 mse; 平均最低者 (此处 Model 1, avg mse = 0.3) 即为最佳, 再用**全部**训练资料重训后用于测试集。

1. 把训练资料切成 N 等份 (例 3 份)
2. 轮流取一份作 validation、其余作 training, 得到 N 种 train/val 配置
3. 每个候选模型在这 N 种配置下都跑一遍, 把结果**平均**
4. 选平均结果最好的模型, 再用**全部训练资料**重新训练, 最后用于测试集

5 回到观看人数预测: Mismatch

5.1 2 月 26 日的预测结果

上一节大家投票多数选了 3 层网络, 就直接用它预测 2021 年的观看人数:

Let's predict no. of views of 2/26!

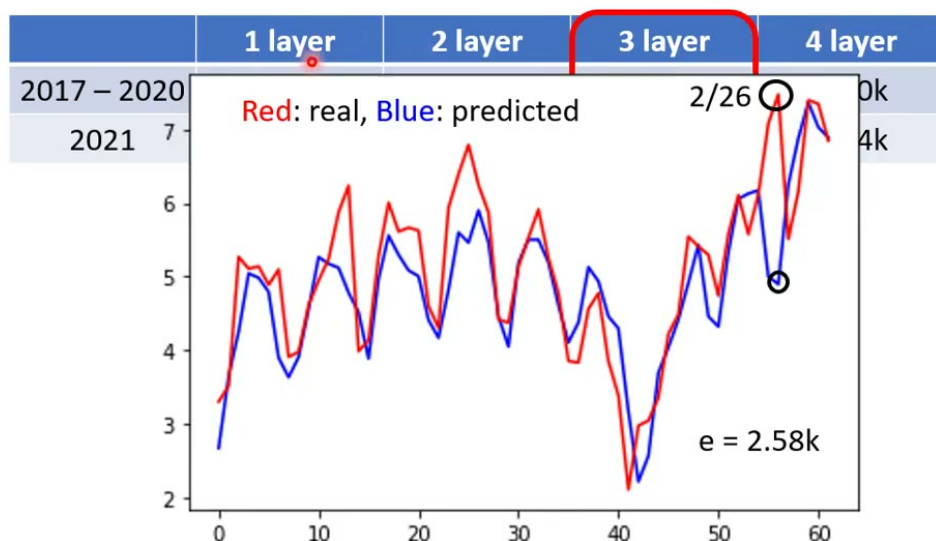


图 11: 3 层网络预测结果: 红线为真实值、蓝线为预测值。模型在 2/26 (2021 年观看人数最高的一天) 预测得非常惨, 误差高达 $e = 2.58k$ 。

老师还跑了 1、2、4 层对比: 2 层、3 层误差都是 2 点多 k, 1 层和 4 层反而好些 (约 1.8k)。但四个模型不约而同都以为 2/26 是个低点——因为根据历史, 礼拜五观看人数通常最少 (“礼拜五晚上大家出去玩了”)。可 2/26 偏偏反常地成了峰值。

这不能怪模型

模型只看得到过去的观看数据, 并不知道 “那天有什么特别”。这种错误既不是 model bias, 也不是单纯的 overfitting, 而是另一种形式——Mismatch。

5.2 Mismatch: 训练与测试分布不同

mismatch

Mismatch

- Your training and testing data have different distributions.

Most HWs do not have this problem, except HW11

Training Data



Simply increasing the training data will not help.

Testing Data



图 12: Mismatch 范例 (作业 11): 训练资料是真实照片, 测试资料却是涂鸦线条画, 两者分布完全不同。此时单纯增加训练资料也无济于事。

Mismatch vs Overfitting

- **Overfitting**: 可以靠增加训练资料来缓解
- **Mismatch**: 训练资料与测试资料的分布本身不一样。此时再增加训练资料也没用——因为多出来的还是“分布不对”的资料

大多数作业都已设计好、不会遇到 mismatch, 唯一的例外是作业 11 (专为 mismatch 而设计: 训练真实照片、测试涂鸦)。

COVID 数据的 mismatch 隐患

以作业一为例: 若用 2020 年资料当训练、2021 年资料当测试, 由于两年背后分布差很多, “怎么训练、什么模型都会铲掉”。所以助教后来改用了别的方式来切分训练/测试资料。

怎么判断是不是 mismatch? 没有公式可套, 要靠你对资料本身、对训练/测试资料产生方式的理解来判断。

6 总结与延伸

本节给出了一套贯穿前期所有作业的**诊断决策树**: 从损失出发, 逐层定位问题、对症下药。

攻略决策树（文字版复盘）

第一步——看训练资料的 loss:

- 大:
 - Model Bias → 把 model 变大（加 feature、加深）
 - Optimization 失败 → 换更好的最佳化方法（下一节）
- 小 → 第二步——看测试资料的 loss:
 - 小 → 通关
 - 大: Overfitting → 增数据 / Data Augmentation / 限制模型; 或 Mismatch → 分布问题, 增数据无用

四种“结果不好”的根因对照

问题	训练 loss	测试 loss	对策
Model Bias	大	大	加大模型（feature/深度）
Optimization 失败	大	大	更好的最佳化（下一节）
Overfitting	小	大	增数据/增强、限制模型
Mismatch	小	大	对齐分布（增数据无效）

注: Model Bias 与 Optimization 失败都表现为“训练 loss 大”, 靠**比较不同模型**来区分; Overfitting 与 Mismatch 都表现为“训练好测试差”, 靠**对资料分布的理解**来区分。

本节关键概念清单

概念	英文	说明
模型偏差	Model Bias	function set 太小, 装不下好函数
最佳化失败	Optimization Issue	好函数存在, gradient descent 找不到
过拟合	Overfitting	训练好、测试差（弹性过大）
资料增强	Data Augmentation	用领域理解从已有资料造新资料
偏差-复杂度权衡	Bias-Complexity Trade-off	测试 loss 随复杂度呈 U 形
验证集	Validation Set	从训练资料切出, 用于挑模型
交叉验证	N-fold Cross Validation	轮流当验证集, 平均后挑模型
公开/私有测试集	Public / Private Testing Set	防止靠刷 public 碰运气
不匹配	Mismatch	训练与测试分布不同

6.1 下一讲预告

- **Optimization 为什么会失败、怎么办**: 局部最小值 (local minima) 与鞍点 (saddle point)
- 之后还会讲到 batch 与 momentum、自动调整学习率、损失函数的选择等“类神经网络训练不起来怎么办”系列

- Regularization、Dropout、CNN 等“限制模型”手段的细节，也会在后续课程展开

本节完 · 下一节：类神经网络训练不起来怎么办（一）——Local Minima 与 Saddle Point

课程视频：<https://www.bilibili.com/video/BV1TAtwzTE1S?p=3>
